



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ Информатика, искусственный интеллект и системы управления

КАФЕДРА Системы обработки информации и управления

ОТЧЕТ

по Лабораторной работе № 5.3

по дисциплине «**МЕТОДЫ МАШИННОГО ОБУЧЕНИЯ**»

Тема: «**Обучение на основе глубоких Q-сетей**»

Студент	<u>ИУ5И-22М</u>	<u></u>	<u>Тавуз Мохамад</u>
	(группа)	(подпись, дата)	(И.О. Фамилия)
Преподаватель		<u></u>	<u>Ю.Е.Гапанюк</u>
		(подпись, дата)	(И.О. Фамилия)

2026 г.

1) Введение

Обучение с подкреплением является одним из направлений машинного обучения, в котором агент обучается принимать решения на основе взаимодействия с окружающей средой. На каждом шаге агент получает наблюдение, выбирает действие, получает вознаграждение и переходит к следующему состоянию. Цель агента состоит в формировании такой стратегии поведения, которая позволяет максимизировать суммарное ожидаемое вознаграждение.

В данной лабораторной работе рассматривается алгоритм Deep Q-Network, или DQN. Этот алгоритм является развитием классического Q-Learning и использует нейронную сеть для аппроксимации Q-функции. Такой подход позволяет применять методы обучения с подкреплением к задачам с большими или сложными пространствами состояний, где использование обычной Q-таблицы становится невозможным или неэффективным.

В качестве среды для проведения экспериментов была выбрана игра Atari Breakout из библиотеки Gymnasium. Для обработки визуальных наблюдений использовалась сверточная нейронная сеть, так как состояние среды представлено в виде игровых кадров. В ходе работы были реализованы основные элементы алгоритма DQN: предобработка кадров, буфер воспроизведения опыта Replay Buffer, сверточная нейронная сеть, целевая сеть Target Network и epsilon-greedy стратегия выбора действий.

2) Описание задания

В соответствии с заданием лабораторной работы необходимо реализовать алгоритм DQN на основе примеров, рассмотренных на лекции. Алгоритм должен использоваться для решения задачи обучения с подкреплением в одной из доступных сред. В качестве среды можно использовать классические среды с полносвязной архитектурой нейронной сети либо игры Atari со сверточной архитектурой нейронной сети.

В данной работе была выбрана среда ALE/Breakout-v5, представляющая игру Atari Breakout. Такой выбор позволяет использовать сверточную архитектуру нейронной сети,

так как наблюдения среды являются изображениями игровых кадров. Это делает задачу более сложной и более близкой к классической постановке алгоритма DQN.

В ходе выполнения лабораторной работы необходимо:

- установить и импортировать необходимые библиотеки;
- создать среду Atari Breakout и вывести ее основные характеристики;
- выполнить предобработку кадров среды;
- реализовать буфер воспроизведения опыта Replay Buffer;
- создать сверточную нейронную сеть DQN;
- создать агента DQN с использованием epsilon-greedy стратегии и целевой сети;
- выполнить обучение агента;
- проверить работу обученного агента на тестовых эпизодах;
- построить график наград при обучении;
- сформировать итоговую таблицу результатов.

3) Текст программы и экранные формы с примерами выполнения

В данном разделе приведены основные этапы программной реализации алгоритма DQN для среды Atari Breakout. Все вычисления были выполнены в среде Google Colab с использованием языка программирования Python и графического ускорителя CUDA.

Для работы со средой обучения с подкреплением использовалась библиотека Gymnasium. Для установки и использования Atari-сред применялись пакеты ale-py и AutoROM. Для численных вычислений использовалась библиотека numpy, для формирования итоговой таблицы — pandas, а для построения графиков — matplotlib.

Для построения и обучения нейронной сети использовалась библиотека PyTorch. В реализации алгоритма DQN были использованы сверточные слои для обработки игровых

кадров, буфер воспроизведения опыта Replay Buffer для хранения переходов агента, а также целевая нейронная сеть Target Network для стабилизации обучения.

```
▶ # Установка и импорт необходимых библиотек для выполнения лабораторной работы

!pip install "gymnasium[atari,accept-rom-license]" ale-py autorom[accept-rom-lice
!AutoROM --accept-license

import gymnasium as gym
import ale_py

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import random
import time
from collections import deque
from tqdm import tqdm

import torch
import torch.nn as nn
import torch.optim as optim

from gymnasium.wrappers import AtariPreprocessing, FrameStackObservation

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

print("Библиотеки успешно импортированы")
print("Версия Gymnasium:", gym.__version__)
print("Версия PyTorch:", torch.__version__)
print("Используемое устройство:", device)
```

```
434.7/434.7 kB 12.9 MB/s eta 0:00:00
Installing build dependencies ... done
Getting requirements to build wheel ... done
Preparing metadata (pyproject.toml) ... done
Building wheel for AutoROM.accept-rom-license (pyproject.toml) ... done
AutoROM will download the Atari 2600 ROMs.
They will be installed to:
  /usr/local/lib/python3.12/dist-packages/AutoROM/roms

Existing ROMs will be overwritten.
Библиотеки успешно импортированы
Версия Gymnasium: 1.3.0
Версия PyTorch: 2.11.0+cu128
Используемое устройство: cuda
```

Рисунок 1 — Установка и импорт необходимых библиотек

3.1. Создание среды Atari Breakout и вывод ее характеристик

На первом этапе была создана среда ALE/Breakout-v5 из набора Atari-сред библиотеки Gymnasium. Данная среда представляет собой игру Breakout, в которой агент управляет платформой и должен отбивать мяч, разрушая блоки в верхней части игрового поля.

После создания среды были выведены ее основные характеристики. Пространство наблюдений представляет собой цветное изображение размером $210 \times 160 \times 3$, где третья размерность соответствует трем цветовым каналам. Пространство действий является дискретным и содержит 4 возможных действия: NOOP, FIRE, RIGHT и LEFT.

Также был выведен начальный кадр среды. Полученная информация показывает, что наблюдения среды имеют визуальный характер, поэтому для их обработки целесообразно использовать сверточную нейронную сеть.

```
▶ # Создание среды Atari Breakout и вывод основных характеристик

env = gym.make("ALE/Breakout-v5", render_mode="rgb_array")

print("Среда:", "ALE/Breakout-v5")
print("Пространство наблюдений:")
print(env.observation_space)

print("\nПространство действий:")
print(env.action_space)

print("\nКоличество действий:", env.action_space.n)

state, info = env.reset()
print("\nРазмер исходного кадра:", state.shape)

action_meanings = env.unwrapped.get_action_meanings()
print("\nСписок действий:")
for i, action_name in enumerate(action_meanings):
    print(i, "-", action_name)

plt.figure(figsize=(5, 5))
plt.imshow(state)
plt.axis("off")
plt.title("Начальный кадр среды Breakout")
plt.show()
```

Рисунок 2 — Код создания среды Atari Breakout

... Среда: ALE/Breakout-v5
Пространство наблюдений:
Box(0, 255, (210, 160, 3), uint8)

Пространство действий:
Discrete(4)

Количество действий: 4

Размер исходного кадра: (210, 160, 3)

Список действий:
0 - NOOP
1 - FIRE
2 - RIGHT
3 - LEFT

Начальный кадр среды Breakout



Рисунок 3 — Результат создания среды Atari Breakout и вывод характеристик

3.2. Предобработка кадров среды Atari Breakout

После создания исходной среды была выполнена предобработка игровых кадров. Данный этап является необходимым при использовании алгоритма DQN для Atari-сред, так как исходные цветные изображения имеют большой размер и содержат избыточную информацию.

Для уменьшения размерности входных данных кадры были преобразованы в оттенки серого, приведены к размеру 84×84 и нормализованы. Также использовался пропуск кадров `frame skip`, позволяющий уменьшить количество обрабатываемых состояний и ускорить обучение агента.

Кроме того, было выполнено объединение четырех последовательных кадров в одно состояние. Такой подход позволяет нейронной сети учитывать динамику движения объектов на экране, например направление движения мяча в игре Breakout. В результате обработанное состояние имеет размер $(4, 84, 84)$, что соответствует четырем последовательным кадрам размером 84×84 .

```
# Предобработка кадров среды Atari Breakout

processed_env = gym.make(
    "ALE/Breakout-v5",
    render_mode="rgb_array",
    frameskip=1
)

processed_env = AtariPreprocessing(
    processed_env,
    frame_skip=4,
    grayscale_obs=True,
    scale_obs=True,
    screen_size=84
)

processed_env = FrameStackObservation(processed_env, stack_size=4)

state, info = processed_env.reset()

print("Размер обработанного состояния:", state.shape)
print("Тип данных:", type(state))
print("Минимальное значение:", np.min(state))
print("Максимальное значение:", np.max(state))

fig, axes = plt.subplots(1, 4, figsize=(12, 3))
for i in range(4):
    axes[i].imshow(np.array(state[i]), cmap="gray")
    axes[i].axis("off")
    axes[i].set_title(f"Кадр {i+1}")

plt.suptitle("Пример обработанных кадров")
plt.show()
```

Рисунок 4 — Код предобработки кадров среды Atari Breakout


```
*** Размер обработанного состояния: (4, 84, 84)
Тип данных: <class 'numpy.ndarray'>
Минимальное значение: 0.0
Максимальное значение: 0.5803922
```

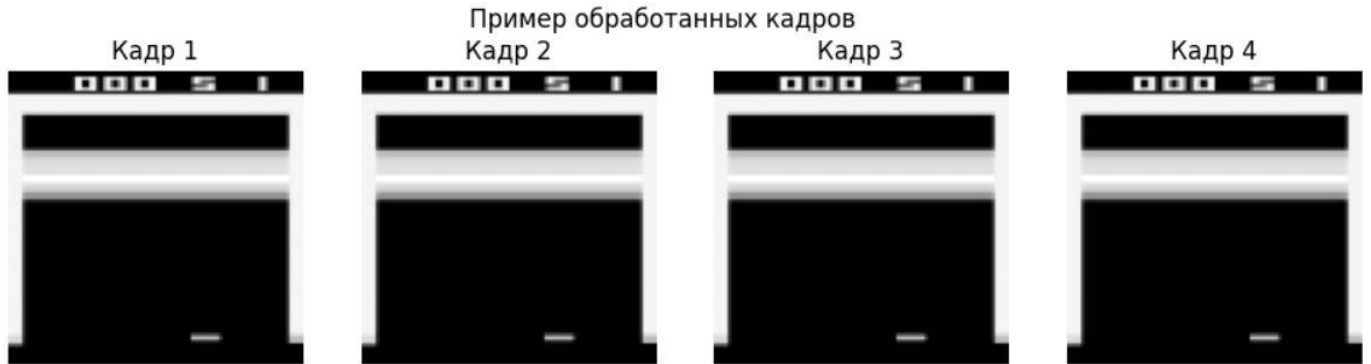


Рисунок 5 — Результат предобработки кадров среды Atari Breakout

3.3. Создание буфера воспроизведения Replay Buffer

На следующем этапе был создан буфер воспроизведения опыта Replay Buffer. Данный элемент является важной частью алгоритма DQN, так как позволяет сохранять переходы агента, полученные во время взаимодействия со средой.

В буфере сохраняются кортежи вида «состояние, действие, награда, следующее состояние, признак завершения эпизода». При обучении нейронной сети из буфера случайным образом выбираются мини-батчи переходов. Такой подход уменьшает зависимость между последовательными наблюдениями и делает процесс обучения более устойчивым.

В данной реализации максимальный размер буфера был задан равным 10000 переходов. Для проверки работы буфера в него был добавлен один пример перехода. В результате текущий размер буфера стал равен 1, а размер сохраняемого состояния составил (4, 84, 84), что соответствует результату предобработки кадров среды Atari Breakout.


```

▶ # Создание буфера воспроизведения опыта Replay Buffer

class ReplayBuffer:
    """
    Буфер воспроизведения опыта для хранения переходов агента.
    """

    def __init__(self, capacity):
        self.buffer = deque(maxlen=capacity)

    def push(self, state, action, reward, next_state, done):
        """
        Добавление одного перехода в буфер.
        """
        self.buffer.append((state, action, reward, next_state, done))

    def sample(self, batch_size):
        """
        Случайная выборка мини-батча из буфера.
        """
        batch = random.sample(self.buffer, batch_size)

        states, actions, rewards, next_states, dones = zip(*batch)

        states = torch.tensor(np.array(states), dtype=torch.float32).to(device)
        actions = torch.tensor(actions, dtype=torch.long).unsqueeze(1).to(device)
        rewards = torch.tensor(rewards, dtype=torch.float32).unsqueeze(1).to(device)
        next_states = torch.tensor(np.array(next_states), dtype=torch.float32).to(device)
        dones = torch.tensor(dones, dtype=torch.float32).unsqueeze(1).to(device)

        return states, actions, rewards, next_states, dones

    def __len__(self):
        """
        Получение текущего размера буфера.
        """
        return len(self.buffer)

replay_buffer = ReplayBuffer(capacity=10000)

state, info = processed_env.reset()
action = processed_env.action_space.sample()
next_state, reward, terminated, truncated, info = processed_env.step(action)
done = terminated or truncated

replay_buffer.push(state, action, reward, next_state, done)

print("Буфер воспроизведения успешно создан")
print("Максимальный размер буфера:", replay_buffer.buffer.maxlen)
print("Текущий размер буфера:", len(replay_buffer))
print("Размер состояния в буфере:", np.array(state).shape)
print("Пример действия:", action)
print("Пример награды:", reward)
print("Признак завершения эпизода:", done)

```

Рисунок 6 — Код создания буфера воспроизведения Replay Buffer

```
... Буфер воспроизведения успешно создан
Максимальный размер буфера: 10000
Текущий размер буфера: 1
Размер состояния в буфере: (4, 84, 84)
Пример действия: 0
Пример награды: 0.0
Признак завершения эпизода: False
```

Рисунок 7 — Результат создания буфера воспроизведения Replay Buffer

3.4. Создание сверточной нейронной сети DQN

После создания буфера воспроизведения была реализована сверточная нейронная сеть DQN, предназначенная для аппроксимации Q-функции. В отличие от табличного Q-Learning, где для каждой пары «состояние–действие» хранится отдельное значение, в DQN Q-значения вычисляются нейронной сетью.

Входом сети является обработанное состояние среды размером (4, 84, 84), то есть четыре последовательных кадра игры Breakout. Архитектура сети содержит три сверточных слоя, которые выделяют признаки из изображения, а затем два полносвязных слоя, формирующих итоговые Q-значения для всех возможных действий.

На выходе сеть формирует тензор размером [1, 4], так как в среде Breakout используется 4 возможных действия. Для стабилизации обучения были созданы две одинаковые сети: основная сеть `policy_net` и целевая сеть `target_net`. В начале работы параметры целевой сети были скопированы из основной сети.



Создание сверточной нейронной сети DQN

```
class DQN(nn.Module):
    """
    Сверточная нейронная сеть для аппроксимации Q-функции.
    """

    def __init__(self, input_channels, num_actions):
        super(DQN, self).__init__()

        self.conv_layers = nn.Sequential(
            nn.Conv2d(input_channels, 32, kernel_size=8, stride=4),
            nn.ReLU(),

            nn.Conv2d(32, 64, kernel_size=4, stride=2),
            nn.ReLU(),

            nn.Conv2d(64, 64, kernel_size=3, stride=1),
            nn.ReLU()
        )

        self.fc_layers = nn.Sequential(
            nn.Linear(64 * 7 * 7, 512),
            nn.ReLU(),
            nn.Linear(512, num_actions)
        )

    def forward(self, x):
        """
        Прямой проход сети.
        """
        x = self.conv_layers(x)
        x = x.view(x.size(0), -1)
        q_values = self.fc_layers(x)
        return q_values

num_actions = processed_env.action_space.n
input_channels = 4

policy_net = DQN(input_channels, num_actions).to(device)
target_net = DQN(input_channels, num_actions).to(device)

target_net.load_state_dict(policy_net.state_dict())
target_net.eval()

sample_state = torch.tensor(np.array(state), dtype=torch.float32).unsqueeze(0).to(device)
sample_q_values = policy_net(sample_state)

print("Сверточная нейронная сеть DQN успешно создана")
print("Количество входных каналов:", input_channels)
print("Количество действий:", num_actions)
print("Размер входного состояния:", sample_state.shape)
print("Размер выходного тензора Q-значений:", sample_q_values.shape)
print("\nАрхитектура сети:")
print(policy_net)
```

Рисунок 8 — Код создания сверточной нейронной сети DQN

```

... Сверточная нейронная сеть DQN успешно создана
Количество входных каналов: 4
Количество действий: 4
Размер входного состояния: torch.Size([1, 4, 84, 84])
Размер выходного тензора Q-значений: torch.Size([1, 4])

Архитектура сети:
DQN(
  (conv_layers): Sequential(
    (0): Conv2d(4, 32, kernel_size=(8, 8), stride=(4, 4))
    (1): ReLU()
    (2): Conv2d(32, 64, kernel_size=(4, 4), stride=(2, 2))
    (3): ReLU()
    (4): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1))
    (5): ReLU()
  )
  (fc_layers): Sequential(
    (0): Linear(in_features=3136, out_features=512, bias=True)
    (1): ReLU()
    (2): Linear(in_features=512, out_features=4, bias=True)
  )
)

```

Рисунок 9 — Результат создания сверточной нейронной сети DQN

3.5. Создание агента DQN

После создания нейронных сетей был реализован агент DQN. Данный агент объединяет основные элементы алгоритма: взаимодействие со средой, выбор действий, сохранение переходов в Replay Buffer и обновление параметров нейронной сети.

Для выбора действий использовалась epsilon-greedy стратегия. В начале обучения значение epsilon равно 1.0, поэтому агент часто выполняет случайные действия и исследует среду. Затем значение epsilon постепенно уменьшается до минимального значения 0.1, что позволяет агенту чаще выбирать действия на основе Q-значений, рассчитанных нейронной сетью.

Обновление сети выполняется на случайных мини-батчах из Replay Buffer. Для каждого перехода вычисляется текущее Q-значение и целевое Q-значение, рассчитанное с использованием целевой сети target_net. Для обучения использовалась функция потерь SmoothL1Loss и оптимизатор Adam. Параметры целевой сети периодически обновлялись путем копирования параметров основной сети policy_net.

▶ # Создание агента DQN

```
class DQNAgent:
    """
    Агент DQN для обучения в среде Atari Breakout.
    """

    def __init__(self, env, policy_net, target_net, replay_buffer,
                 gamma=0.99, lr=0.0001, batch_size=32,
                 epsilon_start=1.0, epsilon_end=0.1, epsilon_decay=5000,
                 target_update=500):

        self.env = env
        self.policy_net = policy_net
        self.target_net = target_net
        self.replay_buffer = replay_buffer

        self.gamma = gamma
        self.batch_size = batch_size
        self.epsilon = epsilon_start
        self.epsilon_start = epsilon_start
        self.epsilon_end = epsilon_end
        self.epsilon_decay = epsilon_decay
        self.target_update = target_update

        self.num_actions = env.action_space.n
        self.optimizer = optim.Adam(self.policy_net.parameters(), lr=lr)
        self.loss_fn = nn.SmoothL1Loss()

        self.steps_done = 0
        self.episodes_reward = []
        self.episodes_steps = []
        self.loss_history = []

    def select_action(self, state):
        """
        Выбор действия на основе epsilon-greedy стратегии.
        """
        self.steps_done += 1

        self.epsilon = self.epsilon_end + (self.epsilon_start - self.epsilon_end) * \
            np.exp(-1.0 * self.steps_done / self.epsilon_decay)

        if random.random() < self.epsilon:
            return self.env.action_space.sample()

        state_tensor = torch.tensor(np.array(state), dtype=torch.float32).unsqueeze(0).to(device)

        with torch.no_grad():
            q_values = self.policy_net(state_tensor)

        return q_values.argmax().item()

    def optimize_model(self):
        """
        Обновление параметров нейронной сети на основе мини-батча из Replay Buffer.
        """
        if len(self.replay_buffer) < self.batch_size:
            return None

        states, actions, rewards, next_states, dones = self.replay_buffer.sample(self.batch_size)

        current_q_values = self.policy_net(states).gather(1, actions)

        with torch.no_grad():
            next_q_values = self.target_net(next_states).max(1)[0].unsqueeze(1)
            target_q_values = rewards + self.gamma * next_q_values * (1 - dones)

        loss = self.loss_fn(current_q_values, target_q_values)

        self.optimizer.zero_grad()
        loss.backward()
        torch.nn.utils.clip_grad_norm_(self.policy_net.parameters(), 10)
        self.optimizer.step()

        self.loss_history.append(loss.item())
        return loss.item()

    def print_info(self):
        """
        Вывод основной информации об агенте.
        """
        print("Агент DQN успешно создан")
        print("Количество действий:", self.num_actions)
        print("Gamma:", self.gamma)
        print("Batch size:", self.batch_size)
        print("Начальное epsilon:", self.epsilon_start)
        print("Минимальное epsilon:", self.epsilon_end)
        print("Target update:", self.target_update)
        print("Размер Replay Buffer:", self.replay_buffer.buffer.maxlen)

dqn_agent = DQNAgent(
    env=processed_env,
    policy_net=policy_net,
    target_net=target_net,
    replay_buffer=ReplayBuffer(capacity=10000)
)

dqn_agent.print_info()
```

Рисунок 10 — Код создания агента DQN

```
... Агент DQN успешно создан
Количество действий: 4
Gamma: 0.99
Batch size: 32
Начальное epsilon: 1.0
Минимальное epsilon: 0.1
Target update: 500
Размер Replay Buffer: 10000
```

Рисунок 11 — Результат инициализации агента DQN

3.6. Обучение агента DQN

После создания агента было выполнено обучение DQN в среде Atari Breakout. В процессе обучения агент взаимодействовал со средой, выбирал действия на основе epsilon-greedy стратегии, сохранял полученные переходы в Replay Buffer и обновлял параметры сверточной нейронной сети.

Обучение выполнялось в течение 30 эпизодов. Для каждого эпизода фиксировались суммарная награда, количество шагов и текущее значение epsilon. Также в процессе обучения периодически обновлялась целевая сеть `target_net`, что позволяет повысить устойчивость обучения и уменьшить колебания целевых Q-значений.

В результате обучение DQN было завершено за 52.641224 секунды. Средняя награда за эпизод составила 1.066667, среднее количество шагов — 186.933333. Финальное значение epsilon составило 0.393182, а количество обновлений функции потерь `loss` составило 5577. Полученные результаты показывают, что была выполнена демонстрационная реализация алгоритма DQN со сверточной архитектурой нейронной сети.

▶ # Обучение агента DQN в среде Atari Breakout

```
def train_dqn_agent(agent, num_episodes=30, max_steps=1000):
    """
    Обучение агента DQN на нескольких эпизодах.
    """
    start_time = time.time()

    for episode in tqdm(range(num_episodes)):
        state, info = agent.env.reset()

        total_reward = 0
        steps = 0
        done = False

        while not done and steps < max_steps:
            action = agent.select_action(state)
            next_state, reward, terminated, truncated, info = agent.env.step(action)
            done = terminated or truncated

            agent.replay_buffer.push(state, action, reward, next_state, done)
            loss = agent.optimize_model()

            state = next_state
            total_reward += reward
            steps += 1

            if agent.steps_done % agent.target_update == 0:
                agent.target_net.load_state_dict(agent.policy_net.state_dict())

        agent.episodes_reward.append(total_reward)
        agent.episodes_steps.append(steps)

        print(
            "Эпизод:", episode + 1,
            "| Награда:", total_reward,
            "| Шаги:", steps,
            "| Epsilon:", round(agent.epsilon, 4)
        )

    training_time = time.time() - start_time

    print("\nОбучение DQN завершено")
    print("Количество эпизодов:", num_episodes)
    print("Время обучения:", training_time)
    print("Средняя награда:", np.mean(agent.episodes_reward))
    print("Среднее количество шагов:", np.mean(agent.episodes_steps))
    print("Финальное значение epsilon:", agent.epsilon)
    print("Количество обновлений loss:", len(agent.loss_history))

    return training_time

training_time = train_dqn_agent(dqn_agent, num_episodes=30, max_steps=1000)
```

Рисунок 12 — Код обучения агента DQN


```
Обучение DQN завершено
Количество эпизодов: 30
Время обучения: 52.641223669052124
Средняя награда: 1.0666666666666667
Среднее количество шагов: 186.93333333333334
Финальное значение epsilon: 0.3931823479304156
Количество обновлений loss: 5577
```

Рисунок 13 — Результат обучения агента DQN

3.7. Проверка обученного агента DQN

После завершения обучения была выполнена проверка обученного агента DQN на тестовых эпизодах. На данном этапе агент выбирал действия жадно, то есть на основе максимальных Q-значений, рассчитанных обученной сверточной нейронной сетью.

Проверка выполнялась на 5 тестовых эпизодах. Для каждого эпизода фиксировались суммарная награда и количество шагов. Такой подход позволяет оценить качество полученной стратегии отдельно от процесса обучения и исследования среды.

По результатам проверки средняя тестовая награда составила 0.4, а среднее количество шагов при тестировании — 568.4. Минимальная награда составила 0.0, максимальная награда — 1.0. Невысокие значения награды объясняются тем, что обучение Atari-агента проводилось в демонстрационном режиме на небольшом количестве эпизодов. При увеличении числа эпизодов качество стратегии может быть улучшено.

▶ # Проверка обученного агента DQN

```
def evaluate_dqn_agent(agent, episodes_count=5, max_steps=1000):
    """
    Проверка качества обученного агента DQN.
    """
    test_rewards = []
    test_steps = []

    old_epsilon = agent.epsilon

    for episode in range(episodes_count):
        state, info = agent.env.reset()

        total_reward = 0
        steps = 0
        done = False

        while not done and steps < max_steps:
            state_tensor = torch.tensor(np.array(state), dtype=torch.float32).unsqueeze(0).to(device)

            with torch.no_grad():
                q_values = agent.policy_net(state_tensor)
                action = q_values.argmax().item()

            next_state, reward, terminated, truncated, info = agent.env.step(action)
            done = terminated or truncated

            state = next_state
            total_reward += reward
            steps += 1

        test_rewards.append(total_reward)
        test_steps.append(steps)

        print(
            "Тестовый эпизод:", episode + 1,
            "| Награда:", total_reward,
            "| Шаги:", steps
        )

    agent.epsilon = old_epsilon

    print("\nПроверка обученного агента DQN завершена")
    print("Количество тестовых эпизодов:", episodes_count)
    print("Средняя тестовая награда:", np.mean(test_rewards))
    print("Среднее количество шагов:", np.mean(test_steps))
    print("Минимальная награда:", np.min(test_rewards))
    print("Максимальная награда:", np.max(test_rewards))

    return test_rewards, test_steps

test_rewards, test_steps = evaluate_dqn_agent(dqn_agent, episodes_count=5, max_steps=1000)
```

Рисунок 14 — Код проверки обученного агента DQN

```
... Тестовый эпизод: 1 | Награда: 0.0 | Шаги: 575
    Тестовый эпизод: 2 | Награда: 0.0 | Шаги: 153
    Тестовый эпизод: 3 | Награда: 1.0 | Шаги: 962
    Тестовый эпизод: 4 | Награда: 0.0 | Шаги: 152
    Тестовый эпизод: 5 | Награда: 1.0 | Шаги: 1000
```

```
Проверка обученного агента DQN завершена
Количество тестовых эпизодов: 5
Средняя тестовая награда: 0.4
Среднее количество шагов: 568.4
Минимальная награда: 0.0
Максимальная награда: 1.0
```

Рисунок 15 — Результат проверки обученного агента DQN

3.8. Построение графика наград при обучении DQN

Для наглядного анализа процесса обучения был построен график суммарных наград, полученных агентом DQN в каждом эпизоде. По оси абсцисс отложен номер эпизода, а по оси ординат — суммарная награда за эпизод.

Так как обучение выполнялось на небольшом количестве эпизодов, значения наград изменяются неравномерно. Для более удобного анализа дополнительно было построено скользящее среднее за 5 эпизодов. Этот показатель позволяет сгладить случайные колебания и оценить общую динамику обучения.

Из графика видно, что награды в процессе обучения остаются невысокими, однако в отдельных эпизодах агент получает положительную награду. Это подтверждает, что реализованный DQN-агент способен взаимодействовать со средой Atari Breakout и получать опыт, однако для получения устойчивой эффективной стратегии требуется значительно большее количество эпизодов обучения.

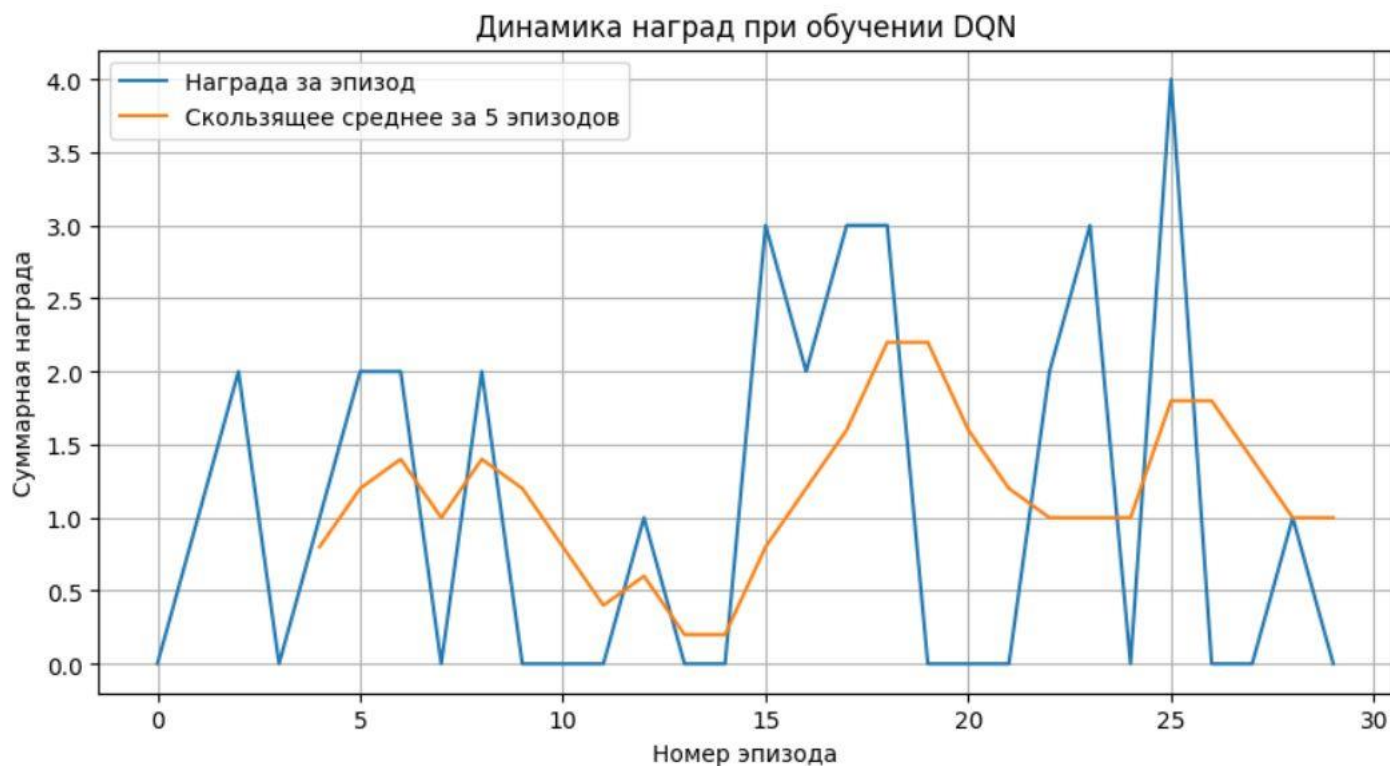


Рисунок 16 — График наград при обучении DQN

3.9. Формирование итоговой таблицы результатов

На заключительном этапе выполнения программы была сформирована итоговая таблица результатов лабораторной работы. В таблицу были включены основные характеристики проведенного эксперимента: выбранная среда, архитектура нейронной сети, размер обработанного состояния, количество действий, количество эпизодов обучения, время обучения, средняя награда при обучении, среднее количество шагов, а также результаты тестирования обученного агента.

Итоговая таблица показывает, что в работе использовалась среда ALE/Breakout-v5 и сверточная архитектура нейронной сети. Размер обработанного состояния составил (4, 84, 84), что соответствует четырем последовательным кадрам игры. Количество возможных действий в среде равно 4.

Обучение агента выполнялось в течение 30 эпизодов и заняло 52.641224 секунды. Средняя награда при обучении составила 1.066667, а среднее количество шагов — 186.933333. На этапе тестирования средняя награда составила 0.4, среднее количество

шагов — 568.4, минимальная тестовая награда — 0.0, максимальная тестовая награда — 1.0.

Показатель		Значение
0	Среда	ALE/Breakout-v5
1	Архитектура нейронной сети	Сверточная нейронная сеть
2	Размер обработанного состояния	(4, 84, 84)
3	Количество действий	4
4	Количество эпизодов обучения	30
5	Время обучения	52.641224
6	Средняя награда при обучении	1.066667
7	Среднее количество шагов при обучении	186.933333
8	Количество тестовых эпизодов	5
9	Средняя тестовая награда	0.4
10	Среднее количество шагов при тестировании	568.4
11	Минимальная тестовая награда	0.0
12	Максимальная тестовая награда	1.0

Рисунок 17 — Итоговая таблица результатов лабораторной работы

4) Заключение

В ходе лабораторной работы был изучен и реализован алгоритм Deep Q-Network для задачи обучения с подкреплением. В качестве среды была выбрана игра Atari Breakout, представленная средой ALE/Breakout-v5 библиотеки Gymnasium. Данная среда использует визуальные наблюдения в виде игровых кадров, поэтому для обработки состояний была применена сверточная архитектура нейронной сети.

В процессе выполнения работы были реализованы основные этапы алгоритма DQN: создание среды, предобработка кадров, формирование состояния из четырех последовательных изображений, создание буфера воспроизведения опыта Replay Buffer, построение сверточной нейронной сети, создание основной и целевой сетей, а также реализация epsilon-greedy стратегии выбора действий.

Обучение агента выполнялось в течение 30 эпизодов. В результате средняя награда при обучении составила 1.066667, а среднее количество шагов — 186.933333. После обучения агент был протестирован на 5 эпизодах. Средняя тестовая награда составила 0.4, минимальная награда — 0.0, максимальная награда — 1.0.

Полученные результаты показывают, что реализованный агент DQN способен взаимодействовать со средой Atari Breakout и обучаться на основе визуальных наблюдений. Невысокие значения награды объясняются ограниченным числом эпизодов обучения, так как полноценное обучение DQN для Atari-игр требует значительно большего времени и вычислительных ресурсов. Таким образом, все поставленные задачи лабораторной работы были выполнены, а использование сверточной нейронной сети позволило реализовать более сложный вариант алгоритма DQN.